

GTK

Part-4

+on Windows

GCC, GTK+ and GLib have ports available on MS Windows. The applications you develop with these three on Linux can easily run on MS Windows, eliminating the need for a rewrite using some other toolkit like MFC or Java Swing. This part of the tutorial goes through the necessary steps to make your GTK+ application compile and execute on MS Windows.

There are different approaches to porting Linux applications to MS Windows. One of the favourite choices is Cygwin.

This is a UNIX/Linux emulation environment that has a Dynamic Link Library that provides a substantial part of the POSIX API, along with an extensive collection of software tools. Cygwin provides GTK+ packages directly under its installation program, but requires X11 as the back-end. This means that users will have to provide an X server as well as the compatibility library provided by Cygwin for their application(s) to work. While this might have its own merits (e.g., complete POSIX compliance), it makes the distribution of ported applications more complicated, requiring inclusion of an X server and Cygwin libraries. The objective of this tutorial is to explore the alternatives to Cygwin,

make porting and distribution easier for the developer, and installation and running easier for the end user.

MinGW/MSYS

MinGW (or Minimalist GNU for Windows) is a native software port of GCC and GNU Binutils. While GCC comes with a compiler and libraries, Binutils has an assembler, linker and other essential tools (like *objcopy*, *ranlib*, *dlltool*, etc). Thus, MinGW provides a complete set of programming tools (including freely distributable Win API headers and libraries) for native Windows application development. The key difference between Cygwin and MinGW is that MinGW does not use a LibC port, but native Windows runtime libraries, thus facilitating native ports. MinGW includes MSYS (or Minimal SYStem), which provides the command interpreter (or shell).